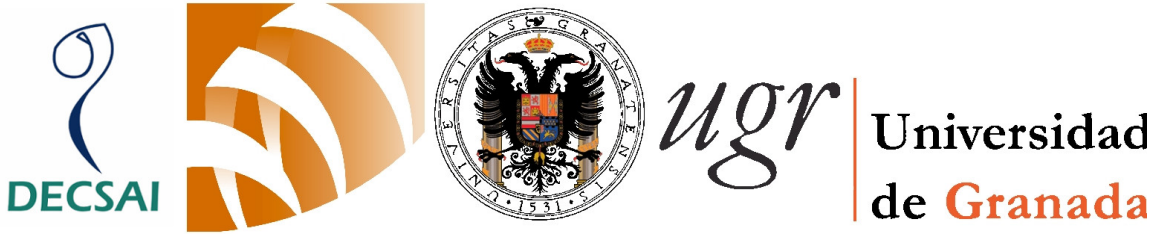
Tema 1: Programación en C++

Fundamentos de Programación

Francisco J. Rodríguez Díaz
(fjrodriguez@decsai.ugr.es)
Departamento de Ciencias de la Computación e I.A.



Universidad de Granada

Tabla de Contenidos I

1. El ordenador, algoritmos y programas
2. Especificaciones de Programas
3. Datos y Tipos de datos
4. Tipos de Datos Simples en C++
5. Operadores y Expresiones

Contenido del tema

1. El ordenador, algoritmos y programas
2. Especificaciones de Programas
3. Datos y Tipos de datos
4. Tipos de Datos Simples en C++
5. Operadores y Expresiones

Componentes de un computador

Figure 1: Hardware: Componentes físicos

Figure 2: Software: Programas

Figure 3: Periféricos

La diferencia entre hardware y software es clara

Programa de ordenador

Algoritmo

Un algoritmo es un secuencia **ordenada** de instrucciones **bien definidas** que **resuelven un problema** concreto.

Programa de ordenador

- Un programa es un *software* que implementa un **algoritmo**.
- **Programar** es el proceso de diseñar, codificar (**implementar**), depurar y mantener un programa.
- La **programación** es la disciplina o acción de **programar**.

Características de un algoritmo

Básicas

- **Correcto:** Sin errores.
- **Precisión:** Sin ambigüedad.
- **Fiable:** Obtiene el mismo resultado con las mismas condiciones.

Esperables

- **Finitud:** Termine en algún momento.
- **Validez:** Resuelve el problema.
- **Eficiencia:** Lo haga en un tiempo aceptable.

Ejemplo de Algoritmo: Receta de macarrones con tomate

Entrada: Ingredientes

- Macarrones.
- Agua.
- Tomate.

Algoritmo

1. Hervir el agua.
2. Añadir macarrones.
3. Esperar mientras no estén blandos.
4. Escurrir agua.
5. Echarle tomate.

Ejemplo de Algoritmo: Receta de macarrones con tomate

Entrada: Ingredientes

- Macarrones.
- Agua.
- Tomate.

Algoritmo

1. Hervir el agua.
2. Añadir macarrones.
3. Esperar mientras no estén blandos.
4. Escurrir agua.
5. Echarle tomate.

¿Es un buen ejemplo de algoritmo?

Ejemplo de Algoritmo: Macarrones con tomate

Entrada: Ingredientes (para dos)

- Macarrones **(300 gramos)**.
- Agua **(medio litro)**.
- Tomate **(medio cartón)**.

Algoritmo

1. Hervir el agua **durante 20 minutos**.
2. Añadir macarrones.
3. Esperar mientras **no estén blandos**.
4. Escurrir agua.
5. Echarle **todo el** tomate.

Ejemplo de Algoritmo: Macarrones con tomate

Entrada: Ingredientes (para dos)

- Macarrones **(300 gramos)**.
- Agua **(medio litro)**.
- Tomate **(medio cartón)**.

Algoritmo

1. Hervir el agua **durante 20 minutos**.
2. Añadir macarrones.
3. Esperar mientras **no estén blandos**.
4. Escurrir agua.
5. Echarle **todo el** tomate.

Debe ser detallado

Datos

Dato

Un **dato** es una **unidad de información** que se representa en un ordenador.

Puede ser

- Nombre (del usuario).
- Valor numérico.
- ...

Los algoritmos trabajan con datos, datos de entrada, de salida, y datos intermedio.

Ejemplo

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4 \cdot a \cdot c}}{2 \cdot a}$$

Ejemplo de algoritmo

Ejemplo

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4 \cdot a \cdot c}}{2 \cdot a}$$

Entrada: a, b, c

Todas las variables son valores decimales.

Algoritmo

1. nueva variable b2 $= b^2$
2. nueva variable resta $= b2 - 4 * a * c$.
3. $x_1 = \dfrac{-b + \sqrt{resta}}{2 * a}$.
4. $x_2 = \dfrac{-b - \sqrt{resta}}{2 * a}$.
5. Devolver x_1 y x_2.

Lenguaje de la Máquina

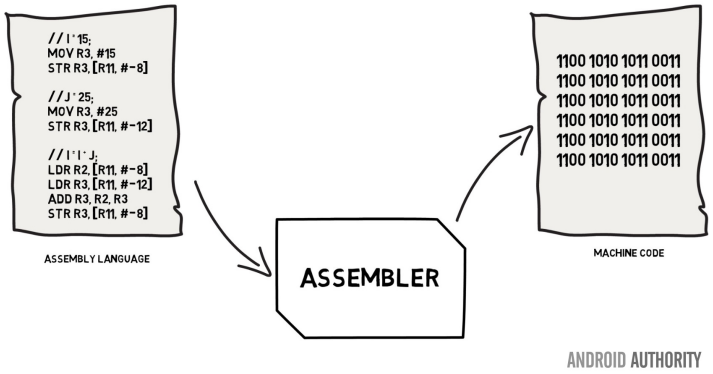
Código Binario

- Los ordenadores sólo entienden código binario.
- Las **instrucciones** son valores binarios.
- Las personas no trabajan en *binario*, sino con palabras.

Lenguajes de Programación

Lenguaje Binario

- Código binario.
- Depende de cada máquina.
- No fácil para persona.



Lenguaje Ensamblador

- No se escribe en binario, sino con palabras especiales.
- Un programa convierte cada palabra en su código binario.
- Mejor, pero mismos problemas (depende del tipo de máquina).

Compilador

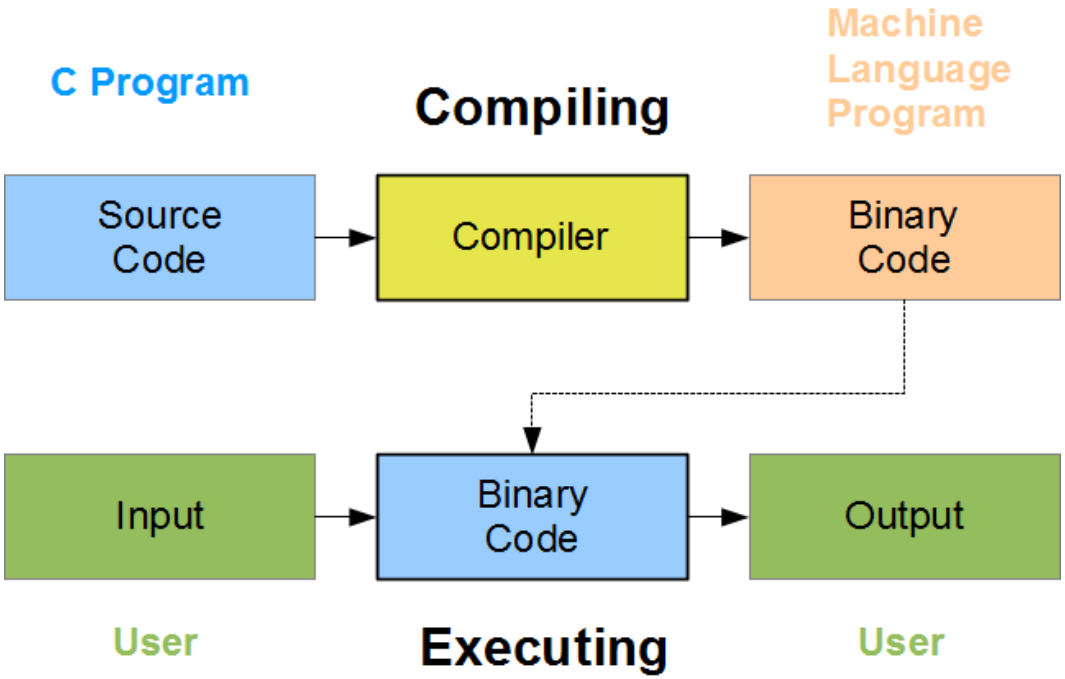
Grace Hooper

- Doctora en matemáticas.
- Militar la marina americana.
- Lideró creación de los primeros compiladores.
- Primer proyecto de Ingeniería del Software.
- Inventó el término *debug*.

Hitos

- **1951**: Primer compilador: lenguaje A-0.
- **1955**: Segundo lenguaje: lenguaje B-0.
- Desarrollo del lenguaje Cobol, aún en uso.

Lenguajes de Programación



Lenguaje de Alto Nivel

- Conjunto de términos y sintaxis concreta.
- Se usa un programa (*compilador*) que lo traduce a binario.
- El usuario puede ejecutar el binario
- **Código fuente** independiente de la arquitectura y Sistema Operativo.

Ejemplo

Fichero main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main(void)
5  {
6    cout << "Bienvenidos a clase\n" <<endl;
7    return 0;
8  }
```

Compilación

```
$ g++ hello.cpp -o hello
```

Ejecución

```
$ ./hello
Bienvenidos a clase
```

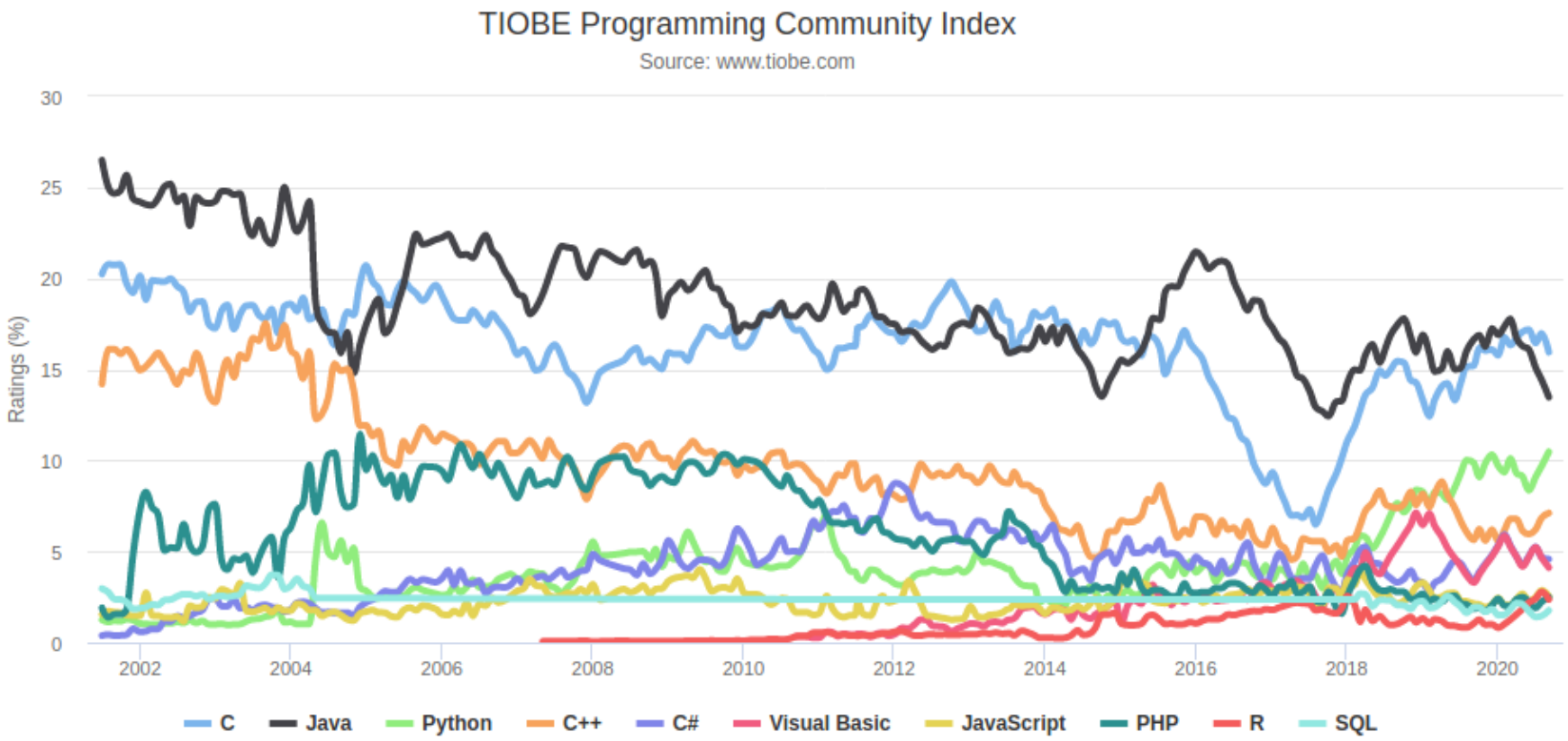
Sobre lenguaje de programación C y C++

Figure 4: Dennis Ritchie y Ken Thompson

Figure 5: Djarne Stroustrup

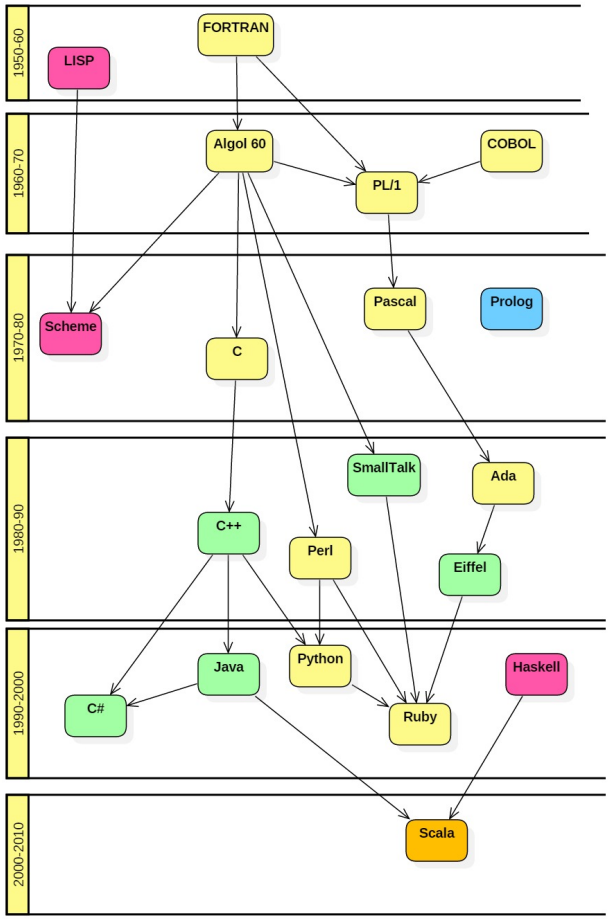
- Veremos C++, que es una versión ampliada del lenguaje C.
- Un programa escrito en C está también escrito en C++.
- C/C++ son muy utilizados e influyentes.

Uso del Lenguaje C++



C y C++ son de los mas utilizados

Influencia del Lenguaje C++



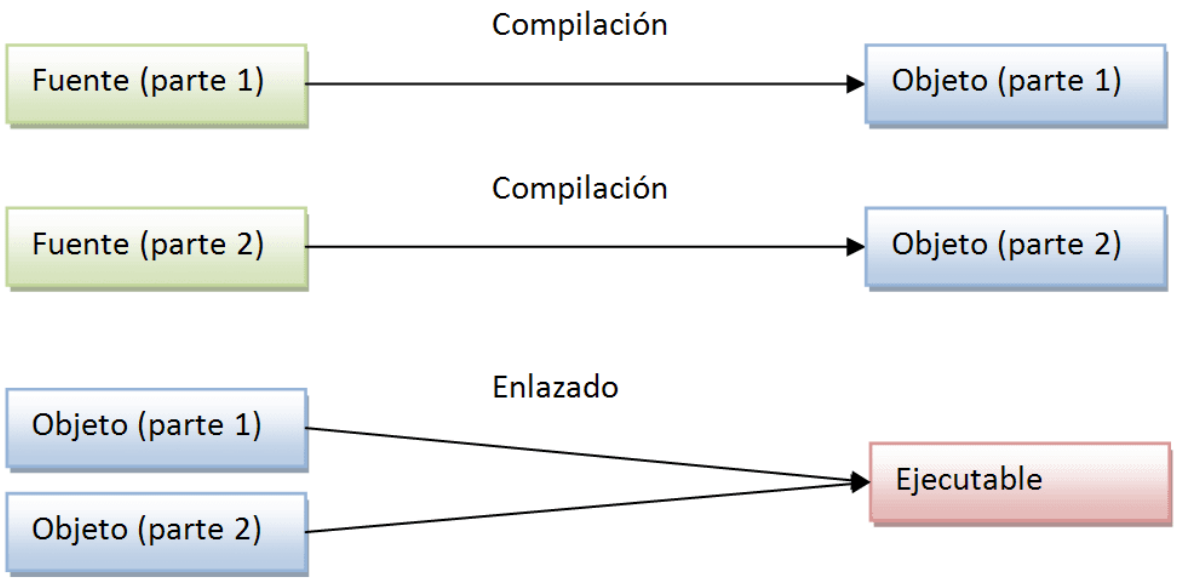
Es un lenguaje muy influyente

Contenido del tema

1. El ordenador, algoritmos y programas
2. Especificaciones de Programas
3. Datos y Tipos de datos
4. Tipos de Datos Simples en C++
5. Operadores y Expresiones

Proceso para realizar un programa

1. Se diseña y se escribe en varios ficheros (**código fuente**).
2. El compilador traduce dichos ficheros en **ficheros objeto** de formato binario.
3. Los ficheros binarios se **enlazan** para formar un **ejecutable**.



Tipos de errores

- El programador crea y edita los ficheros de código fuente según una sintaxis determinada (del lenguaje de programación usado).
- El ordenador hace lo que se le pide paso a paso.
- Los errores son del programador (en el 99% de las veces).

Tipos de errores

- El programador crea y edita los ficheros de código fuente según una sintaxis determinada (del lenguaje de programación usado).
- El ordenador hace lo que se le pide paso a paso.
- Los errores son del programador (en el 99% de las veces).

Tipos de errores

- Compilación
- Ejecución.

Errores de Compilación

Errores de compilación

- No se cumple bien la sintaxis del lenguaje en el código fuente.
- El compilador da **mensaje de error**, no se generan ficheros objeto o ejecutable.
- **Leer los mensajes, ayuda mucho**.

Ejemplo

```
1 int main(void)
2 {
3   cout "Bienvenidos a clase\n" <<endl
4   return 0;
5 }
$ g++ error.cpp -o program
error.cpp: En la función ‘int main()’:
error.cpp:6:42: error: expected ‘;’ before ‘return’
    6 |   cout << "Bienvenidos a clase\n" << endl
      |                                          ^
```

Errores en tiempo de ejecución

Error en tiempo de ejecución

- Se cumple la sintaxis, pero se hace una operación incorrecta.
- El compilador genera el ejecutable (a veces da un aviso).
- Al **ejecutarlo** da el mensaje de error.

Ejemplo de error de tiempo de ejecución

```
1  int main(void) {
2    int x = 3;
3    int y = x/0;
4    return 0;
5  }
$ g++ errorun.cpp -o run
errorun.cpp:3:12: warning: division by zero
   int y = x/0;
            ~^~
$ ./run
Floating point exception (core dumped)
```

Errores lógicos

Errores lógicos

- El programa compila y se ejecuta.
- El programa no hace lo que se espera.
- No hay mensaje de error.

Ejemplo de error lógico

Si queremos calcular la hipotenusa y hacemos:

```
1   lado1 = 4;
2   lado2 = 9;
3   // Un simple error al escribir, debe ser lado2*lado2
4   hipotenusa = sqrt(lado1*lado1+lado2*lado1);
```

Esquema de un programa en C++

holamundo.cpp

```
1 /*
2 * Programa Holamundo.cpp
3 */
4
5 #include <iostream> // Incluimos biblioteca iostream
6
7
8
9 using namespace std; // Usamos espacio de nombres estandar
10
11
12 int main() {
13
14    /* Mostramos Hola, mundo por pantalla */
15    cout << "Hola, Mundo" << endl;
16
17    // Fin del programa
18    return 0;
19 }
20
```

Esquema de un programa en C++

Comentario de varias líneas: /* ... */

Inclusión de bibliotecas:
#include <biblioteca>
ó
#include "biblioteca"

```
1/*
2* Programa Holamundo.cpp
3*/
4
5#include <iostream> // Incluimos biblioteca iostream
6
9using namespace std; // Usamos espacio de nombres estandar
10
11
12int main() {
13
14    /* Mostramos Hola, mundo por pantalla */
15    cout << "Hola, Mundo" << endl;
16
17    // Fin del programa
18    return 0;
19}
20
```

Inclusión de espacio de nombres estándar

Comentario hasta final de línea: // ...

Inicio y fin de programa:

```
int main() {

   return 0;
}
```

Partes de un programa en C++

Comentarios

- El código entre /* y */ lo ignora el ordenador, es para el programador.
- También se puede preceder por // en una misma línea.
- Pueden estar en cualquier parte.

Ejemplo

```
1 /* Este comentario se puede hacer en distintas
2    líneas sin problema.
3 */
4 int x = 640; // inicio el tamaño de la pantalla
```

Partes de un programa en C++

Cabecera

- Se indica funciones externas que se van a usar.
- **Funciones externas** son aquellas que no implementas:
  - Librería externa (hay funciones estándares).
  - Funciones definidas en otros ficheros.

Ejemplo

```
1 #include <iostream> // necesario para escribir por pantalla
2 #include <cmath> // funciones matemáticas (sqrt)
3
4 using namespace std; // luego lo veremos
```

Función main

Función main

- Es la parte del código, instrucciones, que se ejecuta cuando se ejecuta el ejecutable.
- Se llama "*main*" y devuelve un valor numérico (0 si no hay error).

Ejemplo de main

```
1 int main(void) {
2    // Tu código va aquí
3    ...
4    return 0;
5 }
```

Algunos tipos de sentencias

Ejemplos de sentencias

- **Asignación** un valor y guardarlo en una variable.
- **Leer** los datos.
- **Escribir** los datos.

Ejemplos de sentencias

```
1 cin >>variable; // Leer valor de variable del teclado
2 variable = variable + 1; // Modifica la variable
3 cout <<variable; // Escribe variable
```

Separaciones

- Cada instrucción debe terminar con ';'.
- Las instrucciones se pueden agrupar con { y }.

Sobre legibilidad

La legibilidad es importante

- El código fuente debe de ser fácil de leer por cualquier programador.
- Debe ser fácil de modificar/arreglar.

Petición

- Tabular bien.
- Mensajes claros.

Comparando código

```
1   void main() {
2       int valor1, valor2, result;
3
4       cout <<"Primer valor: " <<endl;
5       cin >>valor1;
6       cout <<"Segundo valor: " <<endl;
7       cin >>valor2;
8       cout <<"Resultado: " <<endl;
9       result = valor1*valor2;
10      cout <<result <<endl;
11   }
```

```
1   void main()
2   {
3  int valor1, valor2,
4        result;
5
6   cout <<"Primer valor: " <<endl;
7      cin >>valor1;
8       cout <<"Segundo valor: " <<endl;
9   cin >>valor2;
10
11  cout <<"Resultado: " <<endl;
12  result
13     =
14  valor1
15     *valor2; cout
16    <<result <<endl;
17  }
```

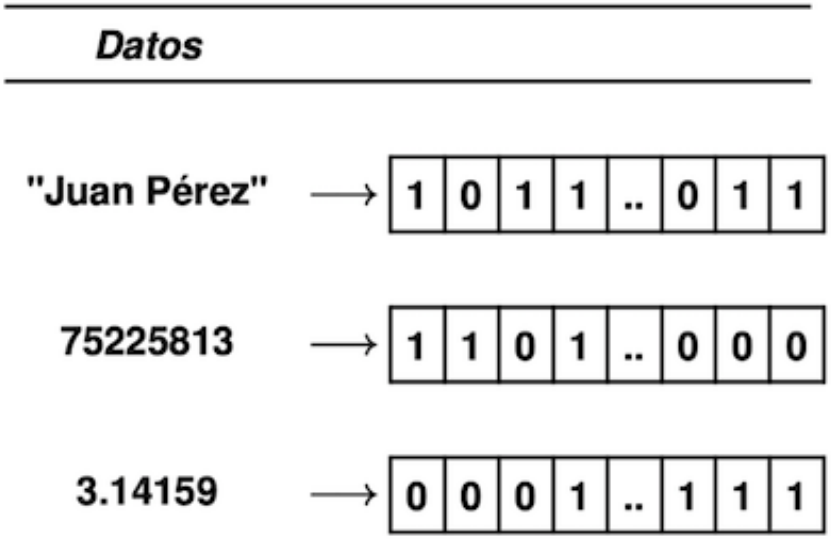
Contenido del tema

1. El ordenador, algoritmos y programas
2. Especificaciones de Programas
3. Datos y Tipos de datos
4. Tipos de Datos Simples en C++
5. Operadores y Expresiones

Datos

Datos

- Los **datos**, como las instrucciones, son **combinaciones de 0 y 1**.
- Al trabajar con un lenguaje de **alto nivel**, no trabajaremos directamente con esa secuencia de 0 y 1, sino **con lo que representan**.
- Cuando **declaramos** un dato, el compilador reserva una **zona de memoria exclusiva** para almacenar ese dato.



Tipos de datos

Tipos de Datos

- Cada lenguaje de programación ofrece sus tipos de datos, denominados **tipos primitivos**.
- En C++, *string*, *int*, *double*, etc.
- Además podremos definir nuestros propios tipos de datos.

Declaración de Datos

- <tipo> <identificador>;

Variables

Variables

- Las variables permiten *guardar* valores de un mismo tipo de dato.
- Deben de declararse (indicar el tipo) antes de poder usarse.
- Su nombre es un identificador.

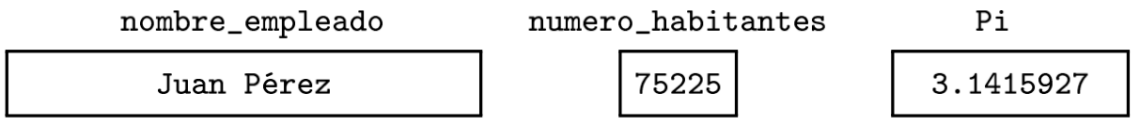
Ejemplo

```
1  int var; // Declaración
2
3  var = 1;
4  cout <<var; // Muestra 1
5  var = 2;
6  cout <<var; // Muestra un 2
7
8  var = 34.5; // ERROR
```

Declaración de Datos

Ejemplo

```
1   string  nombre_empleado;
2   int     numero_habitantes;
3   double  Pi;
4
5   nombre_empleado = "Pedro Ramirez";
6   nombre_empleado = "Juan Pérez"; // Se pierde el antiguo
7   nombre_empleado = 37;           // Error de compilación
8   numero_habitantes = "75225"     // Error de compilación
9   numero_habitantes = 75225;
10  Pi = 3.14156;
11  Pi = 3.1415927;                 // Se pierde el antiguo
```



Declaración de Datos

Identificador

Un identificador es una **secuencia de caracteres alfanuméricos**, que debe de ser **único**.

Restricciones

- Debe **empezar** por una **letra o subrayado** (_).
- **No** puede contener **espacios en blanco** ni ciertos caracteres como **letras acentuadas**, la **letra eñe**, etc.
- Es **sensible** a **mayúsculas y minúsculas**. Los identificadores "*lado*" y "*Lado*" son distintos.
- **No** se puede usar como identificador una **palabra reservada** del lenguaje: **int**, **float**, **if**, **for**, **while**, ...

Declaración de Datos

Facilitar legibilidad

- El identificador debe ser un **reflejo de su contenido**: ~~v~~ -> *voltaje*; ~~l1~~ -> *lado1*
- **No** utilizar **nombres genéricos**: ~~aux~~ -> *copia*; ~~vector1~~-> *calificaciones*
- Usar **minúsculas** para **datos variables**. Las **constantes** en **mayúsculas**. Los **nombres compuestos** se separan con **subrayado**: ~~precioventapublico~~ -> *precio_venta_publico*
- Podemos usar **siglas** como identificadores siempre que sean **ampliamente conocidas**: *pvp*, *tae*
- **Evitar identificadores** que **difieran** en **un solo carácter o en la capitalización**: ~~*cuenta*, *cuentas*, *Cuenta*~~ -> *cuenta*, *coleccion_cuentas*, *cuenta_ppal*

Inicialización de Datos

Inicialización

- Es posible dar un **valor inicial** durante la **declaración**: <tipo> <identificador> = <valor inicial>;
- Un error **lógico** común es usar una variable antes de su inicialización.

Error Lógico

```
1  int main() {
2    double tipo_retencion = 0.18;
3    double salario_bruto;       // valor indeterminado;
4    double total_retencion;     // valor indeterminado
5
6    // Error lógico, salario_bruto tiene valor indeterminado
7    total_retención = salario_bruto * tipo_retencion;
8  }
```

Literales

Literal

Un literal es la especificación de un valor concreto de un tipo de dato

Tipos

- Literales numéricos: 2, 3, 3.1415927.
- Literales de cadenas de caracteres: "Juan Pérez".
- Otros literales: caracteres ('a'), lógicos (true/false).

Constantes

Datos Constantes

- Datos a los que solo permitimos tomar un único valor, fijado de antemano.
- const <tipo> <identificador> = <valor_inicial>
- Identificadores en mayúsculas para diferenciarlos de los datos variables.

Contenido del tema

1. El ordenador, algoritmos y programas
2. Especificaciones de Programas
3. Datos y Tipos de datos
4. Tipos de Datos Simples en C++
5. Operadores y Expresiones

Tipos de Datos Enteros

Representación

- Subconjunto del conjunto matemático **Z**.
- La **cardinalidad** depende del número de **bits** utilizados.
- Distintos tipos enteros. En C++: short, int, long, etc.
- Tipo *int* 32 bits: $[-\frac{2^{32}}{2}, \frac{2^{32}}{2} - 1]$ = [-2 147 483 648, 2 147 483 647]
- Si necesitamos un entero mayor long long [int]. Entero de 64 bits: [-9 223 372 036 854 775 808, 9 223 372 036 854 775 807]

Arithmetic Overflow

```
1  int main() {
2    int entero;
3
4    entero = 2147483647 + 1; // máximo int representable + 1
5
6    cout << entero; // valor indeterminado
7
8  }
```

Tipos de Datos Reales

Representación

- Un dato de tipo real tiene como rango un subconjunto **finito** de $\mathbb{R}$.
- Distintos tipos: float (usualmente 32 bits) y double (usualmente 64 bits).
- Representación **coma flotante**: 42.001 -> valor = 4.2001 escala = 10; 0.42001 -> valor = 4.2001 escala = 10^{-1}.
- El valor se denomina **mantisa** y el coeficiente de la escala **exponente**. En la representación en coma flotante, la base de la escala es 2.
- Por ejemplo 1011 representaría: $1 * \frac{1}{2} + 0 * \frac{1}{2^2} + 1 * \frac{1}{2^3} + 1 * \frac{1}{2^4} = 0.6875$
- Valores como 0.1 o 0.01 no se pueden representar de forma exacta, dando un error de **redondeo**.

Tipos de Datos Reales

Representación

- Poca precisión (número de dígitos consecutivos que pueden representarse) tanto en la parte entera como en la parte real.
- Tipo: float; Tamaño 4 bytes; Rango: +/-3.4 e +/-38; Precisión: 7 dígitos.
- Tipo: double; Tamaño 8 bytes; Rango: +/-1.7 e +/-308: Precisión: 15 dígitos.

Importante

Todas las operaciones realizadas con datos de tipo real pueden devolver valores que solo sean aproximados.

Tipos de Datos Reales

```
1   double valor_real;
2
3   //datos de tipo double con menos de 15 cifras (en decimal)
4
5   valor_real = 11.0; //Almacena 11.0 -> Correcto
6   valor_real = 1.1; //Almacena 1.1000000000000001 --> Problema redondeo
7
8   //datos de tipo double con más de 15 cifras (en decimal)
9
10  valor_real = 100000000000000000100.0;
11  //Almacena 100000000000000000000.0 --> Problema precisión
12
13  valor_real = 0.100000000000000000009;
14  //Almacena 0.10000000000000001 --> Problema precisión
15
16  valor_real = 1.000000000000000000009;
17  // Almacena 1.0 --> Problema precisión
18
19  valor_real = 10000000001.000000000004;
20  // Almacena 10000000001.0 --> Problema precisión
```

Tipo de Datos Cadena

Cadena de Caracteres

Un literal de tipo **cadena de caracteres (string)** es una sucesión de caracteres encerrados entre comillas dobles.

Ejemplo

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      string mensaje_bienvenida;
7      mensaje_bienvenida = "\tFundamentos de Programación\n";
8      cout << mensaje_bienvenida;
9  }
```

Tipo de Datos Cadena

Operaciones con cadenas

Para poder operar con cadenas debemos incluir la biblioteca string

Ejemplo

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string cad;
8      cad = "Hola y ";
9      cad = cad + "adios\n"
10     cout << cad;
11 }
```

Tipo de Datos Cadena

Otras operaciones

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string cadena;
8      int entero;
9      int real;
10
11     entero = 27;
12     cadena = to_string(entero); // Almacena "27"
13
14     cadena = "35";
15     entero = stoi(cadena); // Almacena 35
16
17     cadena = "23.5";
18     real = stod(cadena); // Almacena 23.5
19
20 }
```

Tipo de Datos Caracter

Ejemplo de uso

```
1  char letra_piso;
2
3  letra_piso = 65; //Almacena 65
4  letra_piso = 'A'; //Almacena 65
5  cout << letra_piso << '\n'; // Imprime A
6
7  cin >> letra_piso; // Usuario introduce B
8                     //Almacena 66
9  cout << letra_piso << '\n'; // Imprime B
10
11 letra_piso += 1; // Almacena 67
12 cout << letra_piso << '\n'; // Imprime C
```

Contenido del tema

1. El ordenador, algoritmos y programas
2. Especificaciones de Programas
3. Datos y Tipos de datos
4. Tipos de Datos Simples en C++
5. Operadores y Expresiones

Operador de asignación

Asignación

Sirve para asignar un valor a una variable o constante.

```
Identificador = <valor>; // No olvidar el ;
```

Ejemplos

```
1  int edad;
2  float peso;
3  char letra_dni;
4  bool hombre;
5  double resultado;
6
7  edad = 29;
8  peso = 82.3;
9  letra_dni = 'h';
10 hombre = true;
11 resultado = (3.2+4.0)/2.0;
```

Operador de asignación

Tienen que coincidir ambos tipos

- El tipo de la expresión de la derecha debe de coincidir con la variable de la izquierda.
- No da error, pero ofrece comportamiento extraños.

Ejemplo

```
1   int edad;
2   edad = 34.5;
3   cout <<edad <<endl;
$ g++ operadores2.cpp -Wall -Werror -o op
$ ./op
34
```

Única excepción

Se pueden copiar valores enteros a variables reales (*float* y *double*) sin problema.

Operadores aritméticos

Operaciones matemáticas sencillas

- Se pueden realizar operaciones aritméticas: +, -, *, /, %.
- El operador % sólo está definida con números enteros.

Ejemplos

```
1 num = 5
2 precio = 20.5*num;
3 iva = 0.2;
4 precio_iva = importe+importe*iva;
5 int resto = 5 % 2; // Da 1
6 int otroresto = 6 % 2; // Da 0
7 int ver = 5.2 % 2; // Da error de compilación
```

Cuidado con la División

Cuidado con la división

- Dividir dos números enteros da siempre un entero (ignora decimal).
- Si se divide algún número decimal (o los dos) el resultado es decimal.

```
1 int nota = 8;
2 int total = 10;
3 int intento1 = nota/10; // Da cero
4 int intento2 = nota/total; // Da cero
5 float valor = nota/total; // Da también cero
6 float valorf = nota/10.0; // Da 0.8
7 float valorf = (float nota)/total; // Da 0.8
```

Preferencias de operadores

Distinta prioridad

1. Operadores entre paréntesis.
2. El operador de cambio de signo.
3. Multiplicación (*), división (/) y módulo (%).
4. Sumas (+) y restas (-).

A igual prioridad

- Se resuelven de izquierda a derecha.

Ejemplo

(4+3 * 7)%3%2 * 5 =>

Preferencias de operadores

Distinta prioridad

1. Operadores entre paréntesis.
2. El operador de cambio de signo.
3. Multiplicación (*), división (/) y módulo (%).
4. Sumas (+) y restas (-).

A igual prioridad

- Se resuelven de izquierda a derecha.

Ejemplo

(4+3 * 7)%3%2 * 5 =>
Vale 5.

Cuidado

- a+b/c != (a+b)/2.

Preferencias de operadores

Distinta prioridad

1. Operadores entre paréntesis.
2. El operador de cambio de signo.
3. Multiplicación (*), división (/) y módulo (%).
4. Sumas (+) y restas (-).

A igual prioridad

- Se resuelven de izquierda a derecha.

Ejemplo

(4+3 * 7)%3%2 * 5 =>
Vale 5.

Cuidado

- a+b/c != (a+b)/2.
- En caso de duda usar paréntesis.

Operador de Entrada y Salida

Lectura de una variable

- Se usa con **cin** y el operador **»**.
- Formato: Origen de Datos » variable.

Operador de Entrada y Salida

Lectura de una variable

- Se usa con **cin** y el operador **»**.
- Formato: Origen de Datos » variable.

Escritura de una variable

- Se usa con **cout** y el operador **«**.
- Formato: Origen de Datos » variable.
- **endl** permite especificar nueva línea.

Ejemplos de Entrada y Salida

Lectura de entrada/salida

```
1  int numero;
2
3  cout <<"Introduce un número: ";
4  cin >>numero;
5  numero = numero / 10;
6  cout <<"El numero dividido por diez da " <<numero <<endl;
```

Otro más complejo

```
1  int valor1, valor2, result;
2
3  cout <<"Introduce el primer valor: ";
4  cin >>valor1;
5  cout <<endl <<"Introduce el segundo: ";
6  cin >>valor2;
7  result = valor1+valor2;
8  cout <<"La suma da " <<result <<endl;
```

Otros operadores

Operadores unarios

- Cambio de signo: - (ej: -3).
- Que implican asignación: +=, -=, *=, /=, %=.
- Otros operadores: ++, - -.

Algunos ejemplos

```
1 int number = -3;
2 number++;
3 number--;
4 number += 5;
5 number *= 3;
6 number /= 2;
7 cout <<number <<endl; // ¿Qué vale?
```

Otros operadores

Librerías matemáticas

C++ permite usar muchas funciones matemáticas, como:

- `double sqrt(double)`: Devuelve la raiz cuadrada de un número decimal.
- `double pow(double, double)`: Recibe un par de números decimales, y devuelve el primer número elevado al segundo.
- `int abs(int)`: Devuelve el valor absoluto de un entero.
- `double fabs(double)`: Devuelve el valor absoluto de un decimal.
- ...

Hay que incluir en la cabecera

```
1 #include <cmath>
```

Ejemplos de funciones matemáticas

Ejemplos

```
1 x = 3.7;
2 y = 5;
3 z = pow(x+7, y);
4 k = sqrt(x)+fabs(x);
```

Redondeo

Float a entero

C/C++ permite convertir de flotante a entero con:

- **ceil()** da entero mayor que el número.
- **floor()** da menor entero.
- **round()** da el entero más cercano.

Operador lógico

Datos lógicos (bool)

- Permiten guardar valores verdadero (*true*) o false (*false*).
- Lo usaremos mucho con sentencias condicionales y bucles.
- Tiene sus propios operadores: **&&**, || y **!**.

Operadores lógico Y(&&)

- true && true => true.
- true && false => false.
- false && true => false.
- false && false => false.

Operadores lógico O (||)

- true || true => true.
- true || false => true.
- false || true => true.
- false || false => false.

Operador lógico !

- !true => false.
- !false => true.

Operadores Relacionales

Operadores relaciones

Los datos de un programa se pueden comparar entre sí: $==$, $!=$, $<$, $>$, $<=$, $>=$

Ejemplos

- a $==$ 3 devuelve true (1) si a vale 3, y false (0) en caso contrario
- a $!=$ 3 devuelve true (1) si a NO vale 3, y false (0) en caso contrario
- a $<$ 3 devuelve true (1) si a es menor que 3, y false (0) en caso contrario
- a $<=$ 3 devuelve true (1) si a es menor o igual que 3, y false (0) en caso contrario
- a $>$ 3 devuelve true (1) si a es mayor que 3, y false (0) en caso contrario
- a $>=$ 3 devuelve true (1) si a es mayor o igual que 3, y false (0) en caso contrario

Expresiones Lógicas

Ejemplos

- true && true && false || !false da valor booleano true
- !true || false da valor booleano false
- true && (true || false) da valor booleano true
- true && (!true || false) da valor booleano false
- true && (!true || false) || false da valor booleano false
- (3<7) && (7/3 > 2) devuelve valor false
- ('b'<'a') || (14>2.5) devuelve valor true
- !(3<7) devuelve valor false
- ((8+4<=10) || (8/3 == 2)) && (1!=2) devuelve valor true
- !(((8+4<=10) || (8/3 == 2)) && (1!=2)) devuelve valor false